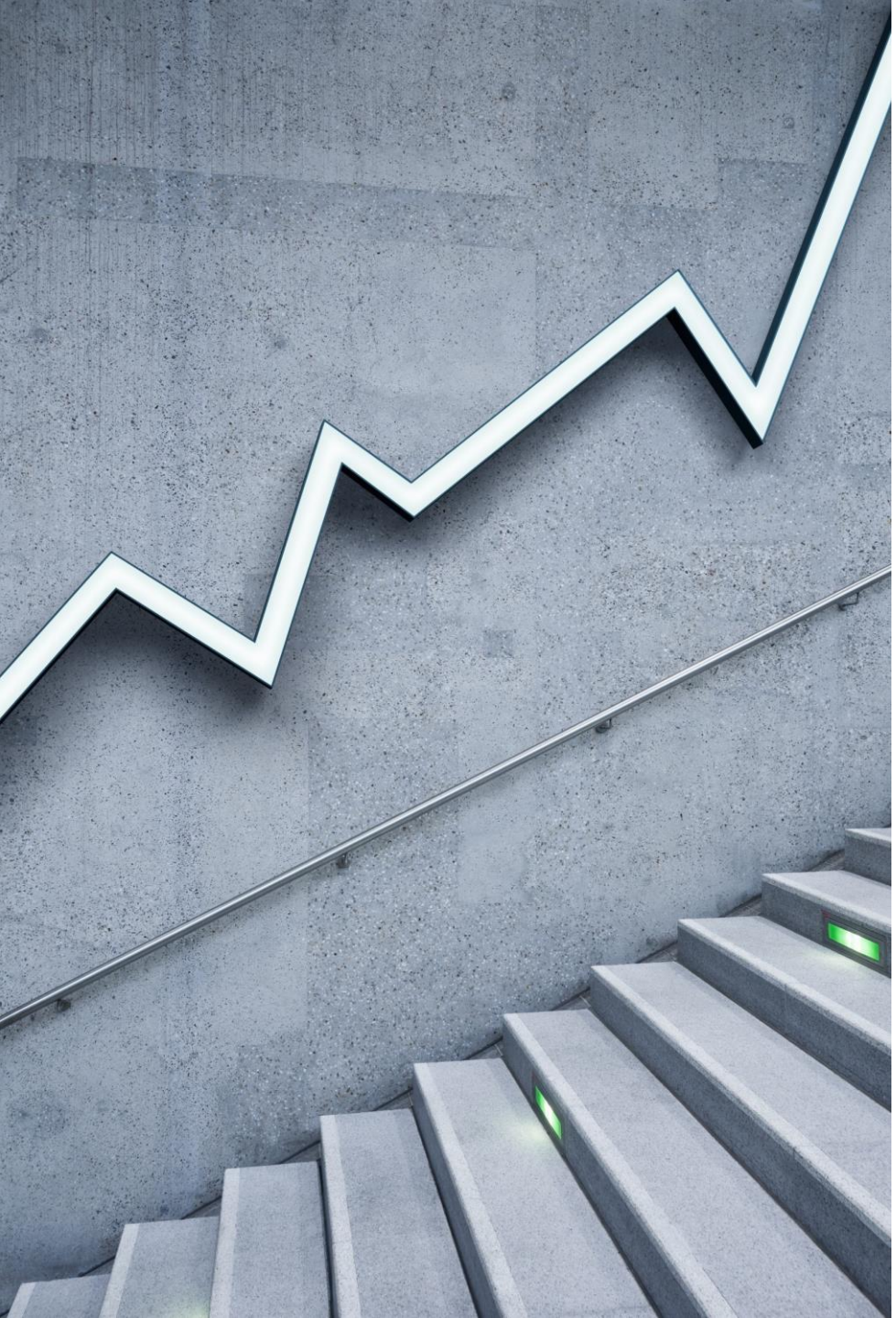


ORM dan Pengelolaan Data pada Laravel

PRAKTIK PEMROGRAMAN BERBASIS WEB II

Materi Pertemuan Ini

ORM dan Pengelolaan Data pada Laravel

1. Konsep ORM
2. Eloquent ORM
3. Model dan Migration
4. Query database
5. Relasi antar-model
6. Pengelolaan data melalui ORM

1. Konsep ORM

Object-Relational Mapping

Teknik yang memetakan tabel basis data menjadi kelas dan objek dalam bahasa pemrograman.

Baris tabel menjadi objek, kolom menjadi properti objek.

Pengembang bekerja dengan objek PHP, bukan menulis perintah SQL secara langsung.

Pemetaan yang terjadi

- Tabel → Kelas Model
- Baris → Objek (instance)
- Kolom → Properti objek
- Relasi antar tabel → Method pada Model

Mengapa Perlu ORM?

Masalah pada akses basis data secara konvensional

Perintah SQL tersebar di banyak berkas sehingga sulit ditelusuri.

Rawan SQL injection bila masukan pengguna tidak dibersihkan.

Perubahan struktur tabel menuntut penyuntingan kueri di banyak tempat.

Kode menjadi terikat pada satu jenis basis data tertentu.

Yang ditawarkan ORM

Akses data terpusat pada Model sehingga mudah dilacak.

Parameter otomatis diamankan oleh framework.

Perpindahan jenis basis data tidak menuntut penulisan ulang kueri.

Kelebihan dan Keterbatasan ORM

Kelebihan

Kode lebih ringkas dan mudah dibaca.

Keamanan dasar terhadap SQL injection sudah ditangani.

Relasi antar tabel ditulis sekali, dipakai berulang.

Perubahan struktur tabel tercatat melalui migration.

Keterbatasan

Menambah lapisan abstraksi sehingga ada beban proses tambahan.

Kueri yang sangat kompleks kadang lebih efisien ditulis langsung sebagai SQL.

Pemakai tetap wajib memahami SQL untuk menelusuri masalah kinerja.

Kueri yang tidak dioptimalkan mudah memicu masalah N+1.

2. Eloquent ORM: Konvensi Penamaan

Eloquent bekerja berdasarkan kesepakatan penamaan

Model Barang otomatis terhubung ke tabel barangs.

Kunci utama diasumsikan bernama id.

Kolom created_at dan updated_at diisi otomatis bila ada.

Kunci tamu diasumsikan bernama nama_model_id, misalnya kategori_id.

Bila penamaan berbeda

- Tetapkan properti $table untuk nama tabel.
- Tetapkan properti $primaryKey untuk kunci utama.
- Tetapkan $timestamps = false bila tabel tidak punya kolom waktu.

Eloquent ORM

Eloquent adalah ORM (Object-Relational Mapping)

- Eloquent adalah ORM (Object-Relational Mapping) yang disediakan oleh Laravel
- Setiap model berhubungan dengan tabel di database
- Eloquent juga mendukung hubungan antar tabel, seperti one-to-many dan many-to-many

Membuat Model dan Menghubungkan ke Tabel

- php artisan make:model NamaModel

```php
class User extends Model {
    protected $table = 'nama_tabel';
}
```

3. Model dan Migration

Dua hal berbeda yang selalu berpasangan

Migration mendefinisikan struktur tabel: nama kolom, tipe data, dan relasinya.

Model mewakili isi tabel tersebut dalam bentuk objek PHP.

Migration mengurus bentuk wadahnya, Model mengurus isinya.

Urutan kerja yang dianjurkan

1. Buat Model beserta migration-nya sekaligus.

2. Tuliskan kolom yang diperlukan pada berkas migration.

3. Jalankan migrate agar tabel terbentuk.

4. Tetapkan $fillable pada Model sesuai kolom yang boleh diisi.

Bila keduanya tidak cocok

Membuat Model dengan Artisan

Perintah dasar

php artisan make:model Barang

Opsi yang sering dipakai

-m membuat migration sekaligus

-c membuat controller sekaligus

-f membuat factory sekaligus

-s membuat seeder sekaligus

--all membuat migration, controller, factory, seeder, dan request sekaligus

Contoh yang paling praktis

php artisan make:model Barang -mcr

Database Migration

Membuat Migration

- php artisan make:migration create_users_table

Mengelola Migration

- php artisan migrate

Rollback dan Reset Migration

- php artisan migrate:rollback
- php artisan migrate:reset

Anatomi Berkas Migration

Dua method utama

up() berisi perubahan yang diterapkan ke basis data.

down() berisi cara membatalkan perubahan tersebut.

Tipe kolom yang sering dipakai

id(), string(), text(), integer(), decimal(), boolean(), date(), timestamps()

foreignId('kategori_id')->constrained() untuk kunci tamu.

Perintah pengelolaan

php artisan migrate menjalankan migration yang belum diterapkan

php artisan migrate:status melihat daftar migration dan statusnya

php artisan migrate:rollback membatalkan kelompok migration terakhir

4. Tiga Cara Mengakses Data

Eloquent ORM

Bekerja dengan objek Model. Paling ringkas dan paling mudah dibaca.

Contoh: Barang::where('stok', '>', 0)->get()

Query Builder

Bekerja langsung pada tabel tanpa Model. Cocok untuk laporan dan agregasi.

Contoh: DB::table('barangs')->select('kategori_id')->groupBy('kategori_id')->get()

Raw SQL

Menulis perintah SQL apa adanya. Dipakai hanya bila dua cara di atas tidak memadai.

Wajib memakai parameter terikat agar tetap aman dari injeksi.

Query Builder

Mengambil Data

- $users = DB::table('users')->get();
- $activeUsers = DB::table('users')->where('active', 1)->get();
- $user = DB::table('users')->where('id', 1)->first();

Menambahkan Data

- DB::table('users')->insert(['name' => 'John Doe',   'email' => 'john@example.com',]);

Mengupdate dan Menghapus Data

- DB::table('users')->where('id', 1)->update(['active' => 0]);
- DB::table('users')->where('id', 1)->delete();

Query Lanjutan: Aggregates, Joins, Subqueries

Aggregates,

- $userCount = DB::table('users')->count();
- $averageAge = DB::table('users')->avg('age');

Joins

- $usersWithPosts = DB::table('users') ->join('posts', 'users.id', '=', 'posts.user_id') ->select('users.name', 'posts.title') ->get();
- $usersWithPosts = DB::table('users') ->leftJoin('posts', 'users.id', '=', 'posts.user_id') ->select('users.name', 'posts.title') ->get();

Subqueries

- $recentUsers = DB::table('users') ->where('created_at', '>', function ($query) { $query->select(DB::raw('DATE_SUB(NOW(), INTERVAL 1 MONTH')); }) ->get();

Optimasi Query Menggunakan Query Builder

Menggunakan Indeks:

Menghindari Select All:

- $users = DB::table('users')->select('name', 'email')->get();

Batch Insert:

- DB::table('users')->insert(
- [['name' => 'John Doe', 'email' => 'john@example.com'],
  - ['name' => 'Jane Doe', 'email' => 'jane@example.com'],]);

Eager Loading:

- $users = User::with('posts')->get();

5. Jenis Relasi pada Eloquent

One to One

hasOne dan belongsTo. Contoh: satu pengguna memiliki satu profil.

One to Many

hasMany dan belongsTo. Contoh: satu kategori memiliki banyak barang.

Many to Many

belongsToMany dengan tabel pivot. Contoh: pengguna dan peran.

Has Many Through

Mengakses relasi melalui tabel perantara.

Polymorphic

morphTo dan morphMany. Satu model dapat berelasi ke beberapa model lain.

Hubungan Lanjutan: One-to-Many, Many-to-Many, Polymorphic

One-to-Many

```php
public function posts() {
    return $this->hasMany(Post::class);
}
```

Many-to-Many

```php
public function roles() {
    return $this->belongsToMany(Role::class);
}
```

Polymorphic

Relasi dan Pengambilan Data

Apa itu Eager Loading?

- $users = User::with('posts')->get();

Apa itu Lazy Loading?

- $user = User::find(1);
- $posts = $user->posts;
- // Ini akan men-trigger query untuk mengambil postingan

Eager Loading dan Masalah N+1

Masalah N+1

Mengambil sepuluh barang lalu memanggil relasi kategori di dalam perulangan

menghasilkan satu kueri untuk daftar barang ditambah sepuluh kueri kategori.

Semakin banyak data, semakin banyak kueri yang dijalankan.

Solusi: eager loading

Barang::with('kategori')->get() hanya menghasilkan dua kueri.

Gunakan with() setiap kali relasi akan dipakai di dalam perulangan.

Cara memeriksa

Pasang Laravel Debugbar untuk melihat jumlah kueri per halaman.

Penggunaan Scopes untuk Query yang Reusable

Apa itu Scopes?

Membuat Scope

Menggunakan Scope

- $activeUsers = User::active()->get();
- $activeUsersWithPosts = User::active()->with('posts')->get();

```php
class User extends Model {
    public function scopeActive($query) {
        return $query->where('active', 1);
    }
}
```

Keuntungan Menggunakan Scopes
1.Reusable:
2.Membaca Lebih Mudah:
3.Organisasi yang Lebih Baik:

6. Pengelolaan Data melalui ORM

Create

Barang::create([...]) atau membuat objek baru lalu memanggil save().

Read

all(), find($id), findOrFail($id), where(...)->get(), first(), paginate(10)

Update

Ambil objeknya, ubah propertinya, lalu panggil save().

Atau langsung: Barang::where(...)->update([...])

Delete

$barang->delete() untuk satu objek.

Barang::destroy($id) bila hanya diketahui kunci utamanya.

Mass Assignment dan Soft Delete

Mass assignment

Pengisian banyak kolom sekaligus dari data formulir.

Laravel menolaknya secara bawaan demi keamanan.

$fillable berisi daftar kolom yang boleh diisi massal.

$guarded berisi daftar kolom yang dilarang diisi massal.

Gunakan salah satu saja, jangan keduanya sekaligus.

Soft delete

Data tidak benar-benar dihapus, hanya ditandai pada kolom deleted_at.

Tambahkan trait SoftDeletes pada Model dan kolom softDeletes() pada migration.

withTrashed() menampilkan data terhapus, restore() mengembalikannya.

Seeding dan Factory

Pengertian dan Manfaat Database Seeding

Manfaat Database Seeding

1. Pengujian Aplikasi:
2. Mempercepat Pengembangan
3. Menjaga Konsistensi Data:
4. Mendemonstrasikan Fitur:

Membuat Factory untuk Generasi Data Dummy

Apa itu Factory?

Membuat Factory

- **Buat Factory Baru:**
  - php artisan make:factory UserFactory
- **Definisikan Struktur Data:**

Menggunakan Factory

- // Membuat satu pengguna
- $user = User::factory()->create();
- // Membuat sepuluh pengguna
- $users = User::factory()->count(10)->create();

```php
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class UserFactory extends Factory {
    protected $model = User::class;

    public function definition() {
        return [
            'name' => $this->faker->name(),
            'email' => $this->faker->unique()->safeEmail(),
            'password' => bcrypt('password'), // Enkripsi password
        ];
    }
}
```

Menggunakan Seeder untuk Pengisian Data Awal

Apa itu Seeder?

Membuat Seeder

- **Buat Seeder Bar**
  - php artisan make:seeder UserSeeder
- Definisikan Data yang Akan Dimasukkan: Buka file

Menjalankan Seeder

- php artisan db:seed --class=UserSeeder
- php artisan db:seed

Menyisipkan Seeder di DatabaseSeede

```php
use Illuminate\Database\Seeder;
use App\Models\User;

class UserSeeder extends Seeder {
    public function run() {
        // Menggunakan factory untuk membuat 10 pengguna
        User::factory()->count(10)->create();
    }
}
```

```php
public function run() {
    $this->call(UserSeeder::class);
}
```

Database Transaction

```php
public function store(StoreTaskRequest $request)
{
    DB::transaction(function () use ($request) {
        $task = Task::create($request->validated());
        // Lakukan operasi lain yang terkait
    });
}
```

Caching Database

Keamanan Database

1. Penggunaan Parameter Binding
2. Validasi Input
3. Menggunakan ORM (Eloquent)
4. Menghindari Query Raw (jika harus maka gunakan binding)
5. Pembaruan berkala (update)

```php
// Menggunakan Query Builder
$user = DB::table('users')->where('email', '=', $email)->first();

// Menggunakan Eloquent
$user = User::where('email', $email)->first();
```

```php
$request->validate([
    'email' => 'required|email',
    'password' => 'required|min:8',
]);
```

```php
DB::select('SELECT * FROM users WHERE email = ?', [$email]);
```

Rangkuman

Enam hal untuk diingat

ORM memetakan tabel menjadi kelas sehingga data diakses sebagai objek.

Eloquent bekerja berdasarkan konvensi penamaan; pahami konvensinya lebih dulu.

Migration membuat perubahan struktur tabel tercatat dan dapat diulang siapa pun.

Tersedia tiga cara akses data: Eloquent, Query Builder, dan Raw SQL.

Relasi ditulis sekali pada Model, lalu dipakai berulang di seluruh aplikasi.

Waspadai masalah N+1 dan selalu batasi kolom yang boleh diisi massal.